

RESEARCH ARTICLE

Quantization-Aware Training With Dynamic and Static Pruning

SANGHO AN^{ID}, JONGYUN SHIN^{ID}, AND JANGHO KIM^{ID}

Department of Computer Science, Kookmin University, Seoul 02707, Republic of Korea

Corresponding author: Jangho Kim (jangho.kim@kookmin.ac.kr)

This work was supported by the “Cooperative Research Program for Agriculture Science and Technology Development” through the Rural Development Administration, Republic of Korea, under Project RS-2024-00332198.

ABSTRACT The evolution of deep neural networks (DNNs) naturally leads to an increase in model size. This necessitates various model compression techniques, such as pruning and quantization, to reduce memory usage and power consumption. In particular, combining these compression techniques can achieve significant cost savings. However, we found that methods using dynamic pruning and quantization suffer from instability in training and poor generalization performance due to the effects of the two Straight Through Estimators (STE). To address this problem, we propose a Quantization-aware training with Dynamic and Static pruning (QADS) method that takes advantage of both pruning and quantization by performing STE operations only during quantization from a certain point in time. In our experiments, the proposed method exhibits more stable training compared to existing techniques and achieves performance improvements on the CIFAR-10/100, ImageNet, and Google Speech Command datasets. The code is provided at <https://github.com/Ahnho/Quantization-aware-training-with-Dynamic-and-Static-Pruning>.

INDEX TERMS Convolutional neural networks, network quantization, network pruning.

I. INTRODUCTION

With the advent of deep neural networks (DNNs), many domains including computer vision, natural language processing, and signal processing have advanced rapidly. Regardless of domains, DNNs have become essential for enhancing the performance in many tasks such as regression, classification and detection. However, as DNNs have evolved the number of parameters has naturally increased, posing challenges for deployment on resource-constrained devices. Consequently, recent years have seen significant advances in techniques like pruning and quantization aimed at compressing deep neural networks.

Network pruning is a compression technique that efficiently compresses models by identifying and removing unimportant weights. This can reduce storage requirements while also improving inference time. The process focuses on removing unnecessary information while maintaining the performance of the model. There are multiple methodologies for network pruning, which can be classified based on the

timing of the pruning: pre-training pruning, during-training pruning, and post-training pruning. Furthermore, pruning methods can be classified as unstructured pruning [1], [2], [3], [4], which entails the removal of individual weights, or structured pruning [5], [6], [7], [8], which involves the elimination of specific architectural components, such as channels, filters, or layers. Our approach falls under the category of unstructured pruning during training.

Network quantization is an effective technique for reducing the size and execution time of deep learning models. By reducing the precision of model weights and activations with fewer bits instead of 32-bit full precision (FP32), it reduces memory space and computational costs. In this process, quantization reduces the bit precision of weight parameters, which decreases storage space and enables hardware-optimized operations, thereby enhancing the model's inference speed.

Quantization can be classified into two primary categories, distinguished by the timing of its application. Two predominant methods for quantization exist, namely Quantization Aware Training (QAT) [9], [10], [11], [12], [13], [14], [15] and Post-Training Quantization (PTQ) [16], [17], [18].

The associate editor coordinating the review of this manuscript and approving it for publication was Andrea De Marcellis^{ID}.

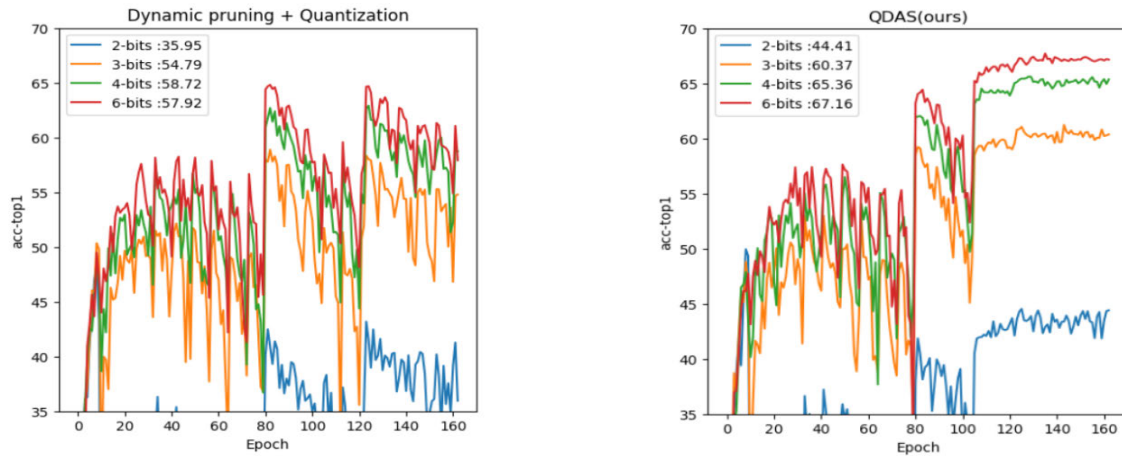


FIGURE 1. The figure on the left depicts the result of a method combining dynamic pruning and quantization on the CIFAR-100 dataset with ResNet32. The figure on the right depicts the result of our method. It can be observed that the graph on the right exhibits greater learning stability and produces higher results than the graph on the left.

Quantization-Aware Training incorporates quantization during the model training process, allowing the training to proceed with quantized weights and thereby minimizing performance degradation due to quantization. This approach ensures that the final quantized model performs comparably to the original model. In contrast, Post-Training Quantization applies quantization after the model has been trained, offering the advantage of quickly performing quantization without additional training.

In this paper, we propose a method that combines pruning and quantization techniques. During the quantization process, we use the LSQ (Learned Step Size Quantization) method [12], one of the QAT techniques that minimizes accuracy loss after quantization. This method achieves higher accuracy and efficiency by learning individually optimized quantization steps for each layer and learning the step sizes during the training process. Additionally, during the quantization process, backpropagation often encounters non-differentiable functions. To address this issue, LSQ uses the Straight-Through Estimator (STE) [19]. STE approximates the gradient of non-differentiable functions during backpropagation, enabling effective learning. Specifically, the STE utilizes the quantization function unchanged during the forward pass, while replacing it with a differentiable linear function during the backward pass to propagate gradients. This approach facilitates effective learning of the quantized values.

For the pruning process, we use dynamic pruning [1], a technique that has been previously studied. Dynamic pruning allows pruning during training and integrates feedback from the pruned model for iterative improvement, creating models with learned sparsity and sparse masks without additional overhead. During the forward pass, this sparse mask is multiplied with the weights. During backpropagation, the mask is not applied, allowing gradients to propagate through the entire model, similar to STE. This approach offers

significant advantages over traditional pruning methods, as it not only improves model efficiency but also enhances performance.

In this paper, we propose a method that combines the advantages of quantization and dynamic pruning to achieve compression while maintaining model performance. However, simply combining these two techniques presents several challenges, such as increased complexity and potential stability issues. For example, as shown in Figure 1, instability can occur during the training process. This instability arises because QAT uses STE to approximate gradients through the non-differentiable quantization operation, while dynamic pruning employs a mask-free approach that produces an STE-like effect, effectively creating a double gradient approximation during backpropagation. These dual approximations amplify estimation errors in the gradient calculation, leading to increasingly inaccurate weight updates and ultimately causing training instability as the errors accumulate over time.

To address the feedback instability issue that arises from the process of combining pruning and quantization, we propose the Quantization-aware training with Dynamic and Static Pruning (QADS). Our methodology optimizes network weights by applying dynamic pruning and quantization during the initial training phase. In this initial phase, we leverage the exploratory benefits of dynamic pruning to discover optimal sparse structures. Subsequently, after the training reaches a certain epoch, we transition from the dynamic approach to static pruning and quantization. With static pruning, gradients only flow through the specific masked weights, eliminating one source of gradient approximation from the backward pass. This reduces the compounding approximation errors that occurred with the dual approximation mechanisms, resulting in more reliable and consistent gradient updates. The controlled gradient flow in static pruning provides a more stable learning environment,

preventing the oscillations and divergence observed when combining QAT with dynamic pruning throughout the entire training process.

In summary, we effectively resolve the feedback instability issue that may occur during the training process by using dynamic pruning to identify sparse masks early in training, followed by static pruning to enhance training stability. Additionally, this approach makes the entire training process more stable and lays the foundation for achieving high performance.

The main contributions of this study are twofold:

1. By alternating between dynamic and static pruning while applying quantization during training, the stability and efficiency of the training process are significantly improved.
2. Stable and superior performance is achieved not only on image datasets but also on speech datasets. On CIFAR-10 and CIFAR-100 datasets, stable and better performance was achieved with the ResNet-32 model, while on the ImageNet dataset, the ResNet-18 model demonstrated outstanding performance. Additionally, on the Google Speech Command dataset, stable and good performance was achieved with the TC-ResNet-18 model for speech data.

II. RELATED WORK

Although deep neural networks offer high accuracy, they often require substantial computational resources. In light of these considerations, a substantial body of research has been dedicated to the development of deep neural network compression technologies. These efforts are aimed at reducing the size of the model, improving computation speed, and enhancing energy efficiency. Among the aforementioned studies, pruning and quantization algorithms represent representative methodologies. This section will present an overview of existing studies related to these technologies, with a view to providing a comprehensive review of the related research in this paper.

Pruning: The process of pruning involves the removal of unimportant neurons (weights), which serves to reduce the overall size of the model while simultaneously enhancing its efficiency. This approach results in a notable improvement in the model's inference speed, a reduction in the required storage space, and a more efficient utilization of computational resources. Many different types of pruning methods have been proposed, each with unique advantages and considerations [20].

Pruning techniques can be broadly classified into three main categories: unstructured pruning [1], [2], [3], structured pruning [5], [6], [7], [8], and semi-structured pruning [21], [22]. The classification largely depends on the pruning structure. Unstructured pruning involves the removal of individual weights, which has the effect of eliminating structural constraints and increasing flexibility. However, this approach is disadvantageous with respect to hardware optimization. In contrast, structured pruning entails the removal of structural elements, such as entire filters or channels. This method is conducive to hardware optimization

and parallelization, but it may be less flexible and potentially less effective.

Depending on when pruning is performed, it can be divided into Pruning before Training [23], [24], Pruning during Training [25], [26], and Pruning after Training [2]. Pruning before training removes unimportant weights at the model design stage. Pruning during training dynamically removes weights during the training process. Pruning after training eliminates unnecessary weights once the model has been fully trained.

According to the pruning criteria, it can be classified as One-shot Pruning [27], [28] and Iterative Pruning [29]. One-shot pruning is a method of lightweighting the model in a single step, while iterative pruning gradually makes the model lighter through multiple pruning and retraining steps.

Additionally, pruning can be categorized into Static Pruning and Dynamic Pruning. Static Pruning applies a sparse mask to all weights without changing it, maintaining fixed sparsity. Dynamic pruning adjusts the importance of weights during the learning process, allowing for more flexibility and adaptation.

Each of these various pruning techniques has its own advantages and disadvantages. Choosing the appropriate pruning technique for specific applications is important. Recently, research into combining pruning with quantization to design more efficient models has been actively progressing. For example, the Pruning-Quantization method aims to further compress and improve model efficiency through both pruning and quantization.

Quantization: Quantization is a technique that compresses a model by converting its parameters to integers or low-precision floating-point numbers. This reduces the model's memory usage and increases inference speed. Depending on when quantization is performed, it can be broadly classified into two types.

The first type is Post-Training Quantization (PTQ) [16], [17], [18]. PTQ quantizes an already trained neural network by converting its weights and activations to low-precision fixed-point numbers. This reduces memory space and increases inference speed. PTQ has the advantage of being able to easily quantize pre-trained models since it does not require additional training. However, there can be a performance degradation during the quantization process.

The second type is Quantization-Aware Training (QAT) [9], [10], [11], [12], [13]. QAT applies quantization during the training process to minimize performance degradation when converting to low-precision fixed-point numbers. QAT methods repeatedly quantize weights and activations during training to optimize the model. One QAT method, Element-Wise Gradient Scaling (EWGS) [13], scales the gradient elements based on the sign and quantization error of the gradient, making the training of quantized networks more stable and accurate. Another method, Learned Step Size Quantization (LSQ) [12], learns the step sizes for quantization, minimizing the impact of quantized values on the network's loss function. While QAT methods maintain

higher accuracy compared to PTQ, they require additional training time.

Pruning + Quantization: The combination of pruning and quantization has been demonstrated to be an effective approach for reducing the size of a model, and is attracting attention as a promising method. The concurrent application of these two techniques allows for a substantial reduction in memory and computational complexity while maintaining model accuracy.

In a notable contribution to this field, Han et al. proposed a three-stage compression pipeline that combines pruning, trained quantization, and Huffman coding [30]. This approach entails the removal of unimportant weights through pruning, the enhancement of the precision of the remaining weights through quantization, and the application of supplementary compression through Huffman coding.

Furthermore, the Optimized Pruning and Quantization (OPQ) method [31] employs only pre-trained weight parameters to perform pruning and quantization in a simultaneous or fine-tuning manner [19]. This method is beneficial for the more efficient compression of pre-trained models.

The Compression via Learned Importance and Pruning-Quantization (CLIP-Q) algorithm [32] proposes a method for performing pruning and quantization in parallel and fine-tuning the model. This algorithm addresses potential errors that may arise during the compression process by leveraging the complementary characteristics of pruning and quantization.

III. METHOD

In this section, we introduce the model compression techniques pruning, and quantization-aware training. We first define static pruning and dynamic pruning, and then quantization-aware training. Finally, we describe our proposed Quantization-aware training with Dynamic and Static Pruning (QADS) method, which utilizes both of these techniques.

A. DYNAMIC AND STATIC PRUNING

In this section, we explain how to remove unnecessary weights using static and dynamic pruning methods. To achieve this goal, we consider a binary mask $\mathbf{M} \in \{0, 1\}^N$ for the number of parameters of N . Based on the importance of the weight magnitude such as L2 norm, we compute the pruned weight $\bar{\mathbf{W}}$ via the original model weight $\mathbf{W} \in \mathbb{R}^N$ and the binary mask $\mathbf{M}$ as follows:

$$\bar{\mathbf{W}} = \mathbf{M} \odot \mathbf{W} \quad (1)$$

where $\odot$ denotes the Hadamard product. It can be written as for a given loss function L , according to the Stochastic Gradient Descent (SGD) [33] update rules.

The gradient of the weights according to the loss function L according to the Stochastic Gradient Descent (SGD) [33] calculation rule is: $\frac{\partial \mathcal{L}}{\partial \bar{w}}$ is the gradient of the loss function L over the pruned weights, and $\frac{\partial \mathcal{L}}{\partial w}$ is the gradient over the

original weights.

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \bar{w}} \frac{\partial \bar{w}}{\partial w} \approx \frac{\partial \mathcal{L}}{\partial \bar{w}} \quad (2)$$

The weight update process is indicated by the pruning methods of both dynamic and static pruning, which employ gradient descent. In static pruning, only the weights masked by $\mathbf{M}$ are updated. If a weight is deemed unimportant during this process, it can be removed. Conversely, in dynamic pruning, the mask $\mathbf{M}$ is not applied, allowing the gradient to flow throughout. The following section introduces weight updates in static pruning and dynamic pruning based on the computational rules of the Stochastic Gradient Descent method (SGD) [33].

Static Pruning Weight Update: Static pruning employs a mask $\mathbf{M}$ in the update process, thereby enabling the determination of the actual weights that will undergo modification. This process guarantees that the weights corresponding to the mask value of 0 are not updated during the forward pass. Consequently, the gradient does not flow in the backward pass for the pruned parts. Accordingly, the learning rate, denoted by η , is employed to update the weights in the following manner:

$$w \leftarrow w - \mathbf{M} \odot \eta \frac{\partial \mathcal{L}}{\partial \bar{w}} \quad (3)$$

Dynamic Pruning Weight Update: The dynamic pruning process does not entail the mask being multiplied by the weight during the forward pass. During the backward pass, the mask is applied so that all gradients can flow through. Then, gradients flow through all weights and the update proceeds.

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial \bar{w}} \quad (4)$$

B. QUANTIZATION AWARE TRAINING

This section presents an explanation of the method of quantizing weights during training, which is conducted using the QAT method. The quantization and de-quantization steps are as follows: Firstly, the model weights w are quantized into an integer value $\tilde{w}_n$, within the range of $[-2^{n-1} + 1, 2^{n-1} - 1]$ based on n -bit. The overall quantization stage can be defined as follows: Subsequently, the step size is defined as $S = \frac{\max(-\min_w, \max_w)}{2^{n-1} - 1}$, where the floating weight range is $[\min_w, \max_w]$. The overall quantization stage can be defined as follows:

where $\lfloor \cdot \rfloor$ is the round operation

$$\tilde{w}_k(w) = \text{Clip}\left(\left\lfloor \frac{w}{S} \right\rfloor, -2^{k-1} + 1, 2^{k-1} - 1\right),$$

$$\text{Clip}(w, x, y) = \begin{cases} x & \text{if } w > x \\ y & \text{if } w < y \\ w & \text{otherwise.} \end{cases} \quad (5)$$

Dequantization stage returns an integer value to the original weight range by multiplying the step-size:

$$\hat{w}_k(w) = \tilde{w}_k(w) \times S. \quad (6)$$

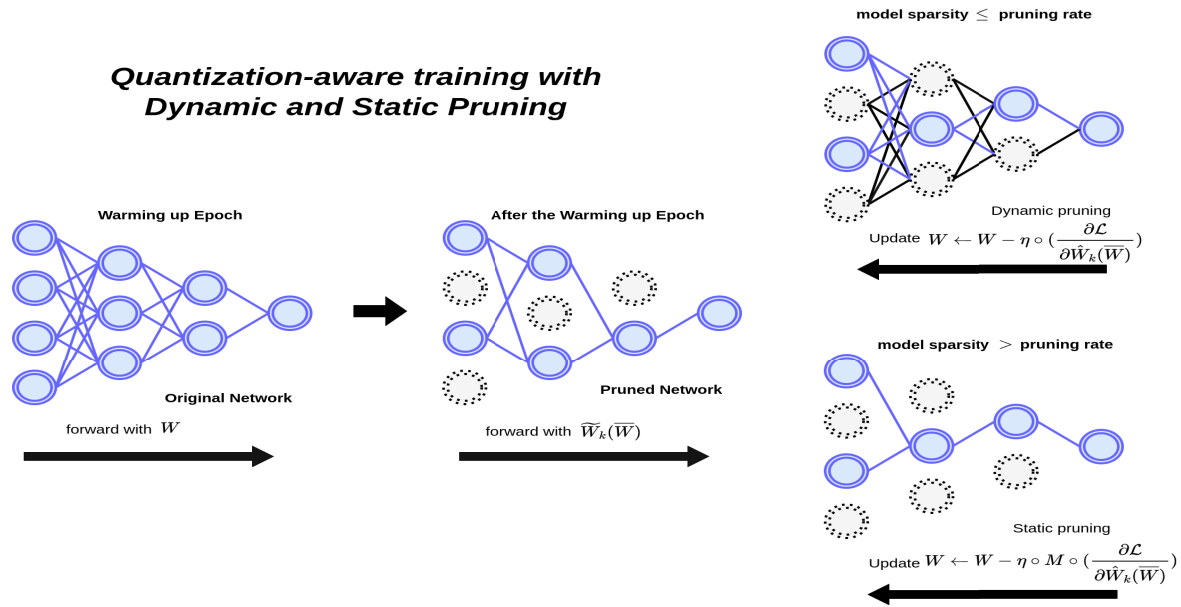


FIGURE 2. The following figure illustrates the comprehensive flow of the QADS algorithm. The graph on the left shows the dense network training during warm-up epochs, and the center image demonstrates the implementation of pruning and quantization procedures after the warm-up epoch. As shown in Eq.(6), our goal is to achieve $\hat{W}_k(\bar{W})$ by inserting the pruned $\bar{W} = M \times W$ instead of $\hat{W}_k(W)$ during the quantization process. The graph on the right shows the backward perspective and formulation for dynamic and static pruning. When the sparsity of the current model is less than or equal to the set pruning ratio, gradients are passed to all weights through dynamic pruning. However, when the model's sparsity exceeds the set pruning ratio, we switch to static pruning to enhance training stability by applying a mask that allows gradients to flow only through selected weights. This transition maintains consistency in weight updates and ensures stability throughout the training process.

since quantization and dequantization processes are non-differentiable, so we utilize a straight-through estimator (STE) [19] for backpropagation. STE approximates the gradient equation is as follows:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{w}_k(w)} \frac{\partial \hat{w}_k(w)}{\partial w} \approx \frac{\partial \mathcal{L}}{\partial \hat{w}_k(w)}. \quad (7)$$

C. QUANTIZATION-AWARE TRAINING WITH DYNAMIC AND STATIC PRUNING (QADS)

In this section, we describe our proposed Quantization-Aware training with Dynamic and Static Pruning (QADS). Our goal is to combine quantization and pruning techniques for effective model compression while maintaining performance. QADS integrates the pruning and quantization methodologies previously described in Sections III-A and III-B, combining the advantages of these compression techniques while addressing the instability that occurs when applying them separately. Specifically, we have designed a training approach that effectively leverages the characteristics of both dynamic and static pruning to work harmoniously with the quantization process.

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{w}_k(\bar{w})} \frac{\partial \hat{w}_k(\bar{w})}{\partial \bar{w}} \frac{\partial \bar{w}}{\partial w} \approx \frac{\partial \mathcal{L}}{\partial \hat{w}_k(\bar{w})}. \quad (8)$$

the $\frac{\partial \mathcal{L}}{\partial \hat{w}_k(\bar{w})}$ denotes the gradient of the loss function L with respect to the quantized and pruned weights, whereas the $\frac{\partial \mathcal{L}}{\partial w}$ represents the gradient with respect to the original weights.

This follows the chain rule as applied in Eq.(2) and Eq.(7) of the pruning and quantization STE, respectively.

The weight update equation for Quantization-Aware Training with Dynamic Pruning is given by the following formula.

$$w \leftarrow w - \eta \circ \left(\frac{\partial \mathcal{L}}{\partial \hat{w}_k(\bar{w})} \right) \quad (9)$$

where we omit the sparse mask so that the gradient flows through all weights and updates them. Next, the weight update equation for Quantization-Aware Training with Static Pruning can be expressed as follows:

$$w \leftarrow w - \eta \circ M \circ \left(\frac{\partial \mathcal{L}}{\partial \hat{w}_k(\bar{w})} \right) \quad (10)$$

The sparse mask employed in the forward process is also incorporated into the update process, ensuring that the gradient flows and updates the weights while excluding the pruned weights. The updates are classified into two categories, Eq.(9) and Eq.(10), based on the target sparsity level. When considered in conjunction with Figure 2, the overall learning process can be observed.

Mask freeze & Mask update We use two methods to observe the results during the pruning process: the mask freeze method, where the mask is frozen midway, and the mask update method, where the mask is continuously updated until the end of training. The mask update continues only until the second decay (123rd epoch), after which the mask is fixed.

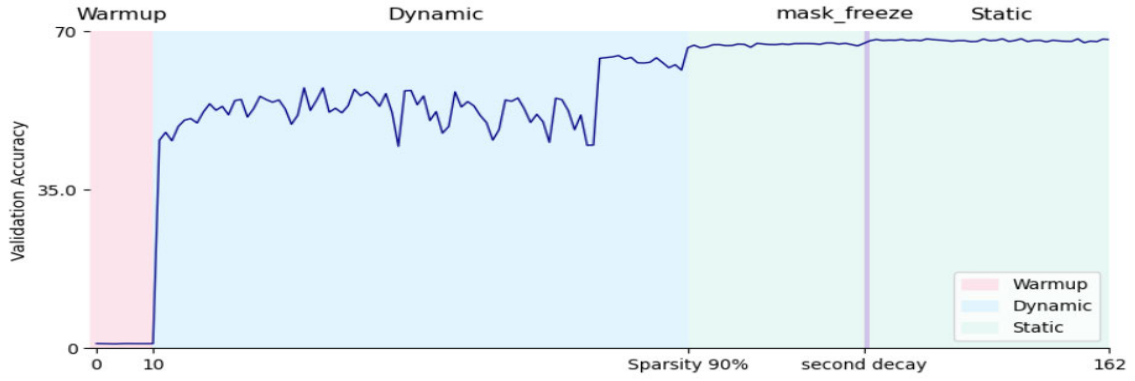


FIGURE 3. The above figure depicts the training process of our QADS method. Quantization is conducted at each epoch following the warm-up phase, and dynamic pruning is transformed into static pruning when 90% sparsity is attained. Furthermore, the mask is fixed during the second decay epoch. The subsequent validation accuracy is illustrated on ResNet-32/CIFAR-100.

Algorithm 1 QADS Algorithm

```

1: Input: Binary mask  $M \in \{0, 1\}^N$ , Binary mask update
   frequency  $F$ , Dense network  $W \in \mathbb{R}^N$ , Pruned network
    $\tilde{W} = M \circ W$ , Pruning Rate PR, Model Sparsity MS,
   Freeze mask epoch FM, stochastic gradient  $g(W) \in \mathbb{R}^N$ 
2: [ Training phase ]
3: for epoch = 1, ...,  $E$  do
4:   if epoch  $\leq 10$  then
5:     Dense Network training {warming up epoch}
6:   end if
7:   if MS  $\leq$  PR then
8:     for Iter = 1, ...,  $T$  do
9:       if Iter %  $F == 0$  and epoch < FM then
10:        Calculate the pruning mask  $M$  using Eq.(2) and
        update the mask every  $F$  iterations.
11:      end if
12:       $W \leftarrow W - \eta \circ g(\tilde{W})$ 
13:    end for
14:   else
15:     for Iter = 1, ...,  $T$  do
16:       if Iter %  $F == 0$  and epoch < FM then
17:        Calculate the pruning mask  $M$  using Eq.(3) and
        update the mask every  $F$  iterations.
18:      end if
19:       $W \leftarrow W - \eta\{M \circ g(\tilde{W})\}$ 
20:    end for
21:   end if
22: end for
23: Output: Pruning mask  $M$ , Model weights  $W$ 

```

IV. EXPERIMENT

The standard models used for comparison of the proposed methods and various algorithms on the CIFAR-10/100 [34], ImageNet [35], and Google Speech Command datasets [36] are ResNet-32/56 [37], ResNet-20 [37], and TC-ResNet-8/14 [38], respectively. These models are used with 2, 3, 4, and 6 bits, respectively. For the ImageNet dataset [35],

experiments were conducted using ResNet-20 with 4-bit quantization.

All experimental results represent the average of five runs, each using different random seeds. The ‘update and freeze’ tag indicates whether the mask was frozen during training. The initial pruning sparsity was set to 90%, with additional sparsity occurring during the quantization process.

A. IMPLEMENTATION DETAILS

We perform training from scratch without finetuning a pre-trained model. During training, we conduct a warming-up phase with the full network for 10 epochs. After the warming-up phase, we quantize all convolutional, linear, and input activation layers. Then, the target sparsity is gradually increased until the second decay. When the model sparsity reaches the predetermined pruning rate, including both pruning and quantization, the process transitions from dynamic pruning to static pruning. The pruning mask update interval is consistently set to 8 iterations. Also, after pruning, the learning rate for the scale weight in the LSQ stage is set to be 100 times smaller than the learning rate for the current model’s weight parameters. The training follows the workflow illustrated in Figure 3. For our experiments, we used a single NVIDIA A6000 GPU for ImageNet dataset training and a single NVIDIA RTX 3090 GPU for all other experiments.

CIFAR Training Schedules: We use the ResNet-32 model with an L2 weight decay of 10^{-4} and an initial learning rate of 0.2. We employ a MultiStep scheduler, reducing the learning rate by a factor of 0.1 at the 80th and 123rd epochs, and continue training up to 162 epochs.

ImageNet Training Schedules: We use the ResNet-18 model with an L2 weight decay of 10^{-4} and an initial learning rate of 0.1. We employ a MultiStep scheduler that reduces the learning rate by a factor of 0.1 at epochs 30, 60, and 80, and continue training until epoch 90.

Google Speech Command Training Schedules: We use the TC-ResNet-18 model with an L2 weight decay of 10^{-4} and an initial learning rate of 0.05. Training is conducted

TABLE 1. Our experiment was conducted to assess the top-1 accuracy of Resnet-32 and Resnet-56 on the CIFAR dataset. The results demonstrated that the method employing the QADS method exhibited the most favorable performance. All quantized methods have the same sparsity rate of 90%. The original method refers to the baseline approach without applying pruning and quantization.

CIFAR-10					
Network	Method	2-bits	3-bits	4-bits	6-bits
Resnet-32	Original	93.53 (32-bits)			
	Static update	78.36 $\pm$ 1.02	87.55 $\pm$ 0.49	89.46 $\pm$ 0.65	89.02 $\pm$ 0.41
	Static freeze	77.31 $\pm$ 2.02	87.81 $\pm$ 0.64	89.34 $\pm$ 0.38	87.65 $\pm$ 0.98
	Dynamic update	81.89 $\pm$ 0.39	88.75 $\pm$ 0.37	90.23 $\pm$ 1.63	91.41 $\pm$ 0.31
	Dynamic freeze	80.23 $\pm$ 1.56	87.53 $\pm$ 0.57	90.39 $\pm$ 0.15	90.63 $\pm$ 0.12
	QADS update (ours)	82.67 $\pm$ 0.27	89.32 $\pm$ 0.17	90.76 $\pm$ 0.37	91.32 $\pm$ 0.25
	QADS freeze (ours)	82.49 $\pm$ 0.41	88.74 $\pm$ 0.78	91.02 $\pm$ 0.09	91.72 $\pm$ 0.23
Resnet-56	Original	94.37 (32-bits)			
	Static update	68.16 $\pm$ 11.14	88.54 $\pm$ 0.05	90.99 $\pm$ 0.23	90.36 $\pm$ 0.07
	Static freeze	78.78 $\pm$ 0.16	89.24 $\pm$ 0.17	90.86 $\pm$ 0.48	90.14 $\pm$ 0.23
	Dynamic update	82.54 $\pm$ 0.78	86.52 $\pm$ 1.82	90.76 $\pm$ 0.92	92.21 $\pm$ 0.15
	Dynamic freeze	81.96 $\pm$ 0.17	88.44 $\pm$ 0.43	89.84 $\pm$ 0.38	92.28 $\pm$ 0.06
	QADS update (ours)	81.61 $\pm$ 1.13	89.21 $\pm$ 0.36	91.09 $\pm$ 0.71	86.84 $\pm$ 5.98
	QADS freeze (ours)	81.69 $\pm$ 0.16	89.65 $\pm$ 0.11	89.72 $\pm$ 1.31	92.71 $\pm$ 0.18
CIFAR-100					
Resnet-32	Original	70.16 (32-bits)			
	Static update	44.18 $\pm$ 0.41	59.02 $\pm$ 0.43	63.03 $\pm$ 0.23	62.46 $\pm$ 1.33
	Static freeze	44.54 $\pm$ 0.84	58.71 $\pm$ 0.44	62.92 $\pm$ 0.57	63.85 $\pm$ 1.11
	Dynamic update	39.13 $\pm$ 1.21	54.45 $\pm$ 0.76	56.41 $\pm$ 1.74	60.46 $\pm$ 0.82
	Dynamic freeze	42.34 $\pm$ 0.74	60.17 $\pm$ 0.59	64.02 $\pm$ 0.07	67.02 $\pm$ 0.29
	QADS update (ours)	45.29 $\pm$ 0.38	60.88 $\pm$ 0.34	64.83 $\pm$ 0.23	66.92 $\pm$ 0.25
	QADS freeze (ours)	44.87 $\pm$ 1.08	61.38 $\pm$ 0.41	65.88 $\pm$ 0.23	68.03 $\pm$ 0.12
Resnet-56	Original	72.63 (32-bits)			
	Static update	42.61 $\pm$ 0.94	60.19 $\pm$ 0.61	65.14 $\pm$ 0.56	63.62 $\pm$ 1.56
	Static freeze	42.91 $\pm$ 0.69	60.26 $\pm$ 0.56	65.21 $\pm$ 0.28	64.31 $\pm$ 0.82
	Dynamic update	40.56 $\pm$ 1.11	56.92 $\pm$ 1.85	63.27 $\pm$ 0.17	65.44 $\pm$ 0.52
	Dynamic freeze	41.68 $\pm$ 1.85	60.62 $\pm$ 1.14	66.17 $\pm$ 1.46	69.58 $\pm$ 0.63
	QADS update (ours)	44.99 $\pm$ 1.31	61.41 $\pm$ 0.99	66.95 $\pm$ 0.33	68.69 $\pm$ 0.52
	QADS freeze (ours)	42.51 $\pm$ 3.04	61.09 $\pm$ 1.03	67.36 $\pm$ 0.23	69.81 $\pm$ 0.32

TABLE 2. Our experiment was conducted to assess the top-1 accuracy of Resnet-18 on the ImageNet dataset. The results demonstrated that the method employing the QADS method exhibited the most favorable performance. All methods have the same sparsity rate of 90 %. The original method refers to the baseline approach without applying pruning and quantization.

ImageNet			
Network	Method	Accuracy (%)	Bits
Resnet-18	Original	69.7	32
	Static update	48.33	4
	Static freeze	49.87	
	Dynamic update	63.13	
	Dynamic freeze	63.77	
	QADS update (ours)	64.01	
	QADS freeze (ours)	64.54	

using a MultiStep scheduler, reducing the learning rate by a factor of 0.1 at the 80th and 123rd epochs, continuing up to 200 epochs.

B. EXPERIMENT RESULTS

In this study, we present the experimental results of the ResNet-32 and ResNet-56 models on the CIFAR dataset

and the TC-ResNet model on the Google Speech Command dataset. The performance of the QADS-update and QADS-freeze methods is analyzed and compared with the existing static pruning + QAT and dynamic pruning + QAT methods.

CIFAR Dataset Results: As illustrated in Table 1, the CIFAR-10 dataset, the QADS methods demonstrated superior performance with both ResNet-32 and ResNet-56 models. For ResNet-32, QADS-update achieved 82.67% and 89.32% accuracy with 2 and 3 bits respectively, while QADS-freeze reached a peak accuracy of 91.72% at 6 bits. With ResNet-56, QADS-update attained 91.07% accuracy-bits, and QADS-freeze achieved 92.7% at 6 bits. For the more challenging CIFAR-100 dataset, QADS-freeze consistently outperformed other methods. Using ResNet-32, it achieved accuracies of 61.38%, 65.88%, and 68.03% at 3, 4, and 6 bits respectively. With ResNet-56, QADS-freeze reached 67.36% and 69.81% accuracy at 4 and 6 bits. In all cases, both QADS methods surpassed the performance of existing quantization and pruning techniques.

Google Speech Command Dataset Results: As illustrated in Table 3 QADS methods also excelled on the Google Speech Command dataset. Using the TC-ResNet-8 model,

TABLE 3. Our experiment was conducted to assess the top-1 accuracy of TC-Resnet-8 and TC-Resnet-14 on the Google Speech Command dataset. The results demonstrated that the method employing the QADS method exhibited the most favorable performance. All methods have the same sparsity rate of 90 %. The original method refers to the baseline approach without applying pruning and quantization.

Network	Method	Google Speech Command			
		2-bits	3-bits	4-bits	6-bits
TC-Resnet-8	Original	96.1 (32-bits)			
	Static update	92.93 $\pm$ 0.11	93.78 $\pm$ 0.17	76.69 $\pm$ 33.66	70.34 $\pm$ 22.98
	Static freeze	92.83 $\pm$ 0.11	93.92 $\pm$ 0.12	93.43 $\pm$ 0.14	66.89 $\pm$ 33.19
	Dynamic update	92.91 $\pm$ 0.22	94.66 $\pm$ 0.15	94.76 $\pm$ 0.22	94.91 $\pm$ 0.21
	Dynamic freeze	93.16 $\pm$ 0.25	94.62 $\pm$ 0.15	94.63 $\pm$ 0.11	94.89 $\pm$ 0.15
	QADS update (ours)	90.64 $\pm$ 0.72	94.21 $\pm$ 0.18	94.51 $\pm$ 0.14	94.61 $\pm$ 0.12
	QADS freeze (ours)	94.38 $\pm$ 0.06	95.27 $\pm$ 0.09	95.38 $\pm$ 0.16	95.3 $\pm$ 0.11
TC-Resnet-14	Original	96.2 (32-bits)			
	Static update	93.22 $\pm$ 0.16	93.51 $\pm$ 0.62	89.91 $\pm$ 1.23	71.12 $\pm$ 13.95
	Static freeze	93.05 $\pm$ 0.14	80.42 $\pm$ 22.54	62.69 $\pm$ 32.31	83.26 $\pm$ 10.64
	Dynamic update	94.29 $\pm$ 0.06	95.27 $\pm$ 0.06	95.56 $\pm$ 0.04	95.52 $\pm$ 0.23
	Dynamic freeze	94.08 $\pm$ 0.31	95.37 $\pm$ 0.15	95.46 $\pm$ 0.14	95.49 $\pm$ 0.13
	QADS update (ours)	94.13 $\pm$ 0.37	95.42 $\pm$ 0.17	95.65 $\pm$ 0.14	95.64 $\pm$ 0.15
	QADS freeze (ours)	94.63 $\pm$ 0.11	95.59 $\pm$ 0.05	95.51 $\pm$ 0.15	95.7 $\pm$ 0.03

QADS-freeze achieved remarkable consistency across different bit-widths, with an overall accuracy of 94.38%. It reached 95.27% accuracy at 2 bits, 95.38% at 3 bits, and 95.3% at both 4 and 6 bits.

ImageNet Dataset Results: As illustrated in Table 2, QADS methods also excelled on the ImageNet dataset. Using the ResNet-18 model, QADS-freeze achieved higher accuracy than other methods at 4-bits, reaching 64.54%.

These results demonstrate the robustness and effectiveness of the QADS approach, particularly the freeze variant, in maintaining high accuracy even at low bit-widths. Across all experiments on CIFAR, ImageNet, and Google Speech Command datasets, QADS methods consistently outperformed existing quantization and pruning techniques, showcasing their potential for efficient model compression while preserving accuracy.

A comprehensive review of the experimental results shows that performance is often higher when the pruning mask is fixed in the middle of the training process. Fixing the pruning mask increases the structural stability of the model. By maintaining a consistent structure, the model can focus on fine-tuning weights within this stable system, contributing to overall training stability and performance improvement. In addition, maintaining a consistent weight distribution helps reduce information loss during the quantization process, allowing for more efficient model training.

Conversely, if the pruning mask is updated until the end of training, performance may be higher with 2-bit and 3-bit quantization. This is because the range of values available to the model during quantization is very limited. In such an environment, continuously evaluating and adjusting the importance of the weights is critical to encoding information efficiently. By continuously updating the mask during training, the model can be fine-tuned to make the most of its limited expressiveness, preserving important weights and

removing less significant ones, thus optimizing performance even at lower bit rates.

The role of the pruning mask during training has a significant impact on model performance and training efficiency. Proper management of the pruning mask allows the model to use limited resources effectively while achieving high performance. This approach is particularly important in resource-constrained environments, as it enhances the usability and applicability of the model. Therefore, selecting and applying the appropriate strategy for freezing and updating the pruning mask is crucial.

Our analysis in this study shows that the QADS method consistently outperforms existing static and dynamic pruning + QAT methods on the CIFAR and Google Speech Command datasets. This suggests that the QADS method is highly effective for deep learning model optimization.

V. CONCLUSION

In this paper, we propose a compression method called Quantization-aware training with Dynamic and Static Pruning (QADS). The proposed method combines dynamic and static pruning techniques. After a warm-up epoch in a dense network, the training continues with both pruning and quantization. To address the training instability that arises when applying pruning and quantization simultaneously, we perform dynamic pruning after the warm-up epoch and switch to static pruning at epochs where a certain sparsity level is reached, thereby improving training stability. For instance, the proposed method can be applied to the CIFAR-10 dataset. When using the ResNet-32 architecture, we observed that our method achieves higher accuracy. Specifically, continuous mask updating yielded higher accuracy at lower bit levels (2-3 bits), while fixing the mask mid-training yielded better performance at higher bit levels (4-6 bits). Overall, combining both dynamic and

static pruning with quantization resulted in better outcomes compared to using a single pruning method alone. This indicates that the proposed compression method can enhance the performance and efficiency of neural networks.

REFERENCES

- [1] T. Lin, S. U. Stich, L. Barba, D. Dmitriev, and M. Jaggi, "Dynamic model pruning with feedback," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2020.
- [2] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," 2018, *arXiv:1803.03635*.
- [3] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," 2018, *arXiv:1810.05270*.
- [4] J. Kim, J. Yoo, Y. Song, K. Yoo, and N. Kwak, "Finding efficient pruned network via refined gradients for pruned weights," in *Proc. 31st ACM Int. Conf. Multimedia*, Oct. 2023, pp. 9003–9011.
- [5] J.-H. Luo, J. Wu, and W. Lin, "ThiNet: A filter level pruning method for deep neural network compression," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 5058–5066.
- [6] Y. He, G. Kang, X. Dong, Y. Fu, and Y. Yang, "Soft filter pruning for accelerating deep convolutional neural networks," 2018, *arXiv:1808.06866*.
- [7] M. Lin, R. Ji, Y. Zhang, B. Zhang, Y. Wu, and Y. Tian, "Channel pruning via automatic structure search," 2020, *arXiv:2001.08565*.
- [8] M. Lin, R. Ji, Y. Wang, Y. Zhang, B. Zhang, Y. Tian, and L. Shao, "HRank: Filter pruning using high-rank feature map," 2020, *arXiv:2002.10179*.
- [9] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," 2017, *arXiv:1712.05877*.
- [10] R. Gong, X. Liu, S. Jiang, T. Li, P. Hu, J. Lin, F. Yu, and J. Yan, "Differentiable soft quantization: Bridging full-precision and low-bit neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 4852–4861.
- [11] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," 2018, *arXiv:1806.08342*.
- [12] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," 2019, *arXiv:1902.08153*.
- [13] J. Lee, D. Kim, and B. Ham, "Network quantization with element-wise gradient scaling," 2021, *arXiv:2104.00903*.
- [14] J. Kim, S. Chang, and N. Kwak, "PQK: Model compression via pruning, quantization, and knowledge distillation," 2021, *arXiv:2106.14681*.
- [15] J. Kim, "Quantization robust pruning with knowledge distillation," *IEEE Access*, vol. 11, pp. 26419–26426, 2023.
- [16] M. Nagel, R. Ali Amjad, M. van Baalen, C. Louizos, and T. Blankevoort, "Up or down? Adaptive rounding for post-training quantization," 2020, *arXiv:2004.10568*.
- [17] E. Park, S. Yoo, and P. Vajda, "Value-aware quantization for training and inference of neural networks," 2018, *arXiv:1804.07802*.
- [18] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," 2021, *arXiv:2106.08295*.
- [19] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.
- [20] H. Cheng, M. Zhang, and J. Q. Shi, "A survey on deep neural network pruning-taxonomy, comparison, analysis, and recommendations," 2023, *arXiv:2308.06767*.
- [21] X. Ma, F.-M. Guo, W. Niu, X. Lin, J. Tang, K. Ma, B. Ren, and Y. Wang, "PCONV: The missing but desirable sparsity in DNN weight pruning for real-time execution on mobile devices," 2019, *arXiv:1909.05073*.
- [22] F. Meng, H. Cheng, K. Li, H. Luo, X. Guo, G. Lu, and X. Sun, "Pruning filter in filter," 2020, *arXiv:2009.14410*.
- [23] H. Tanaka, D. Kunin, D. L. K. Yamins, and S. Ganguli, "Pruning neural networks without any data by iteratively conserving synaptic flow," 2020, *arXiv:2006.05467*.
- [24] C. Wang, G. Zhang, and R. Grosse, "Picking winning tickets before training by preserving gradient flow," 2020, *arXiv:2002.07376*.
- [25] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," 2017, *arXiv:1708.06519*.
- [26] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li, "Learning structured sparsity in deep neural networks," 2016, *arXiv:1608.03665*.
- [27] N. Lee, T. Ajanthan, and P. H. S. Torr, "SNIP: Single-shot network pruning based on connection sensitivity," 2018, *arXiv:1810.02340*.
- [28] J. Frankle, G. K. Dziugaite, D. M. Roy, and M. Carbin, "Pruning neural networks at initialization: Why are we missing the mark?" 2020, *arXiv:2009.08576*.
- [29] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2015, pp. 1135–1143.
- [30] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, *arXiv:1510.00149*.
- [31] P. Hu, X. Peng, H. Zhu, M. M. S. Aly, and J. Lin, "OPQ: Compressing deep neural networks with one-shot pruning-quantization," in *Proc. AAAI Conf. Artif. Intell.*, May 2021, vol. 35, no. 9, pp. 7780–7788.
- [32] F. Tung and G. Mori, "CLIP-Q: Deep network compression learning by in-parallel pruning-quantization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7873–7882.
- [33] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proceedings of COMPSTAT'2010*. Springer, 2010, pp. 177–186.
- [34] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," Tech. Rep., 2009.
- [35] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.
- [36] P. Warden, "Launching the speech commands dataset," Google Res. Blog, Tech. Rep., 2017.
- [37] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [38] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan, "PACT: Parameterized clipping activation for quantized neural networks," 2018, *arXiv:1805.06085*.



SANGHO AN received the B.S. degree in software from Kookmin University, in 2024, where he is currently pursuing the master's degree with the Department of Computer Science.



JONGYUN SHIN received the B.S. degree in electronics from Kookmin University, in 2023, where he is currently pursuing the master's degree with the Department of Computer Science.



JANGHO KIM received the B.S. degree in information and communication engineering from Dongguk University, in 2015, the M.S. degree in computer science and engineering from Pohang University of Science and Technology (POSTECH), in 2017, and the Ph.D. degree from the Department of Transdisciplinary Studies, Seoul National University, in 2022. He is currently an Assistant Professor with the School of Artificial Intelligence, Kookmin University. His research interests include computer vision, machine learning, and deep learning applied to the image.

...